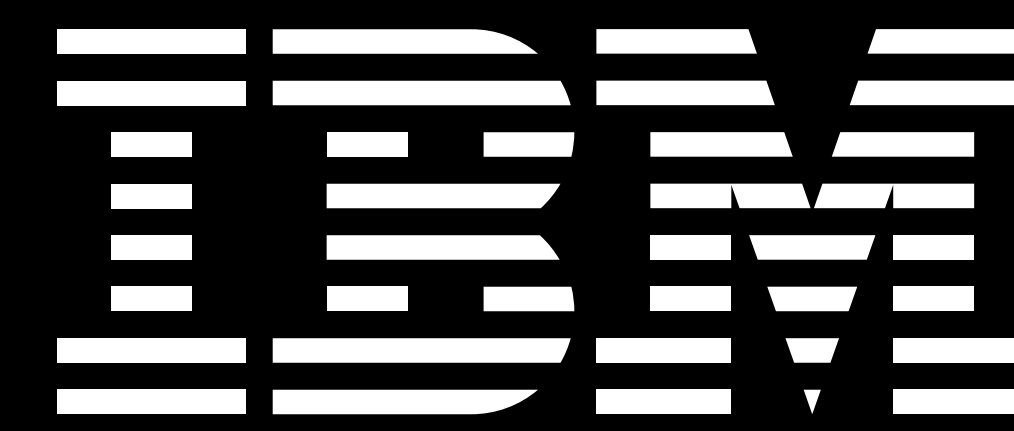


Context-Parallel Mamba2: Scaling to 1M+ Tokens



Garrett Goon

Mamba2: A Linear Attention Layer

The quadratic $\mathcal{O}(\text{seq_len}^2)$ scaling of Transformers attention has increasingly become a bottleneck as context lengths have ballooned with the advent of reasoning models and rising prevalence of modalities such as video and audio.

Architecture	Prefill/Train	Prefill/Train
Attention	$\mathcal{O}(\text{seq_len}^2)$	$\mathcal{O}(\text{seq_len})$
Mamba2	$\mathcal{O}(\text{seq_len})$	$\mathcal{O}(1)$

Mamba2 [1] is an alternative information propagation algorithm belonging to the steadily growing class of *linear* $\mathcal{O}(\text{seq_len})$ attention mechanisms [2]. Its GPU-aware design enables hardware utilization comparable to quadratic attention during training, and reduces the decoding time and cache-space from $\mathcal{O}(\text{seq_len})$ to $\mathcal{O}(1)$. See [3], [4], [5], [6] for a partial list of models which utilize Mamba2

Mamba2 Architecture (Simplified)

Mamba2 relies on two central mechanisms for propagating information:

1. Short 1D causal convolutions [7]
2. A gated recursion relation

Giving inputs $x_{sd} \in \mathbb{R}^{\text{seq_len} \times \text{d_model}}$, the Mamba2 outputs are schematically

$$z_{sd} \sim \text{gated_recursion}(\text{causal_conv1d}(x_{sd}))$$

Both components require adaptation in the CP implementation.

Causal Convolutions

The causal convolution [7] is a depthwise, 1D convolution along the sequence dimension with a short filter, typically of width $K = 4$:

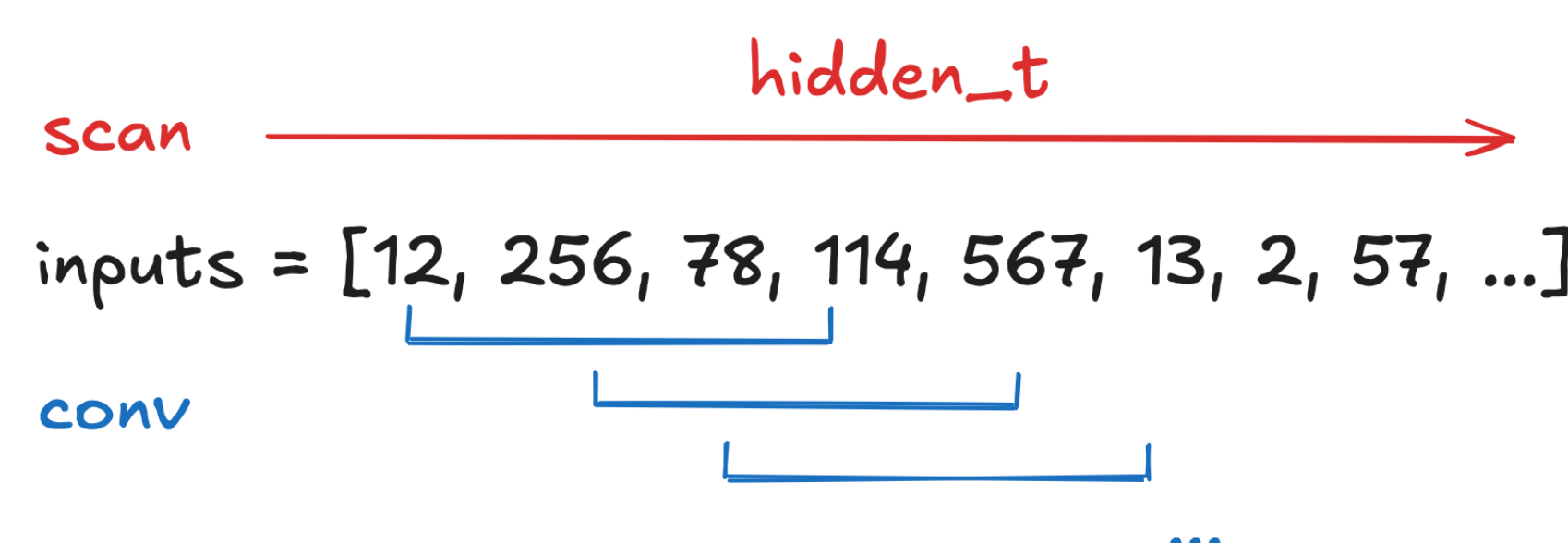
$$z_{sd} = \sum_{k=0}^K W_{kd} x_{(s-k)d}.$$

Gated Recursion Relations

The central elements in the Mamba2 recursion relation are of the form:

$$z_{sd} = e^{-A_s} z_{(s-1)d} + \Delta_s x_{sd}$$

where the data-dependent $A_s, \Delta_s \geq 0$ control the deletion and addition of information to the state z_{sd} . While the complete tensor z_{sd} can be constructed in $\mathcal{O}(\text{seq_len})$ by solving the recursion in the naive manner, such an approach is suboptimal in practice as it cannot leverage GPU tensor cores. For this reason, the recursion is solved using a chunked, matmul-based strategy which has inferior theoretical scaling, but superior in-practice wall time [1].



Context Parallelism

Leveraging Mamba2's long-context scaling advantages requires long-context training which poses engineering challenges as $\text{activation_mem} \propto \text{seq_len}$.

Context-parallelism (CP), in which sequences are sharded along the sequence dimension, is a natural and scalable approach to long-sequence training.

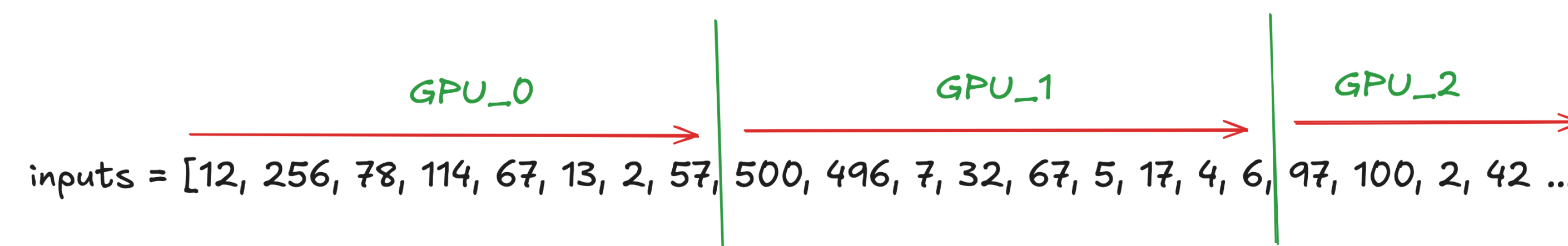


Figure 2: Context Parallel Sharding

CP Convolutions

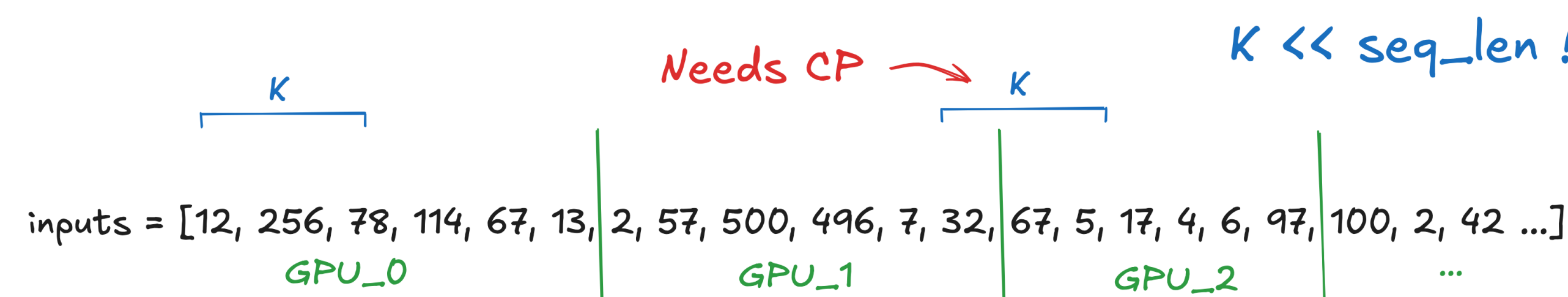


Figure 3: Context Parallel Causal Convolution

Only minimal modifications required for CP causal convolutions. Each GPU handles $C = \text{seq_len} / \text{cp_degree}$ tokens, and only the outputs for the first $K - 1 \ll C$ tokens require communication for computational correctness.

An efficient algorithm is as follows:

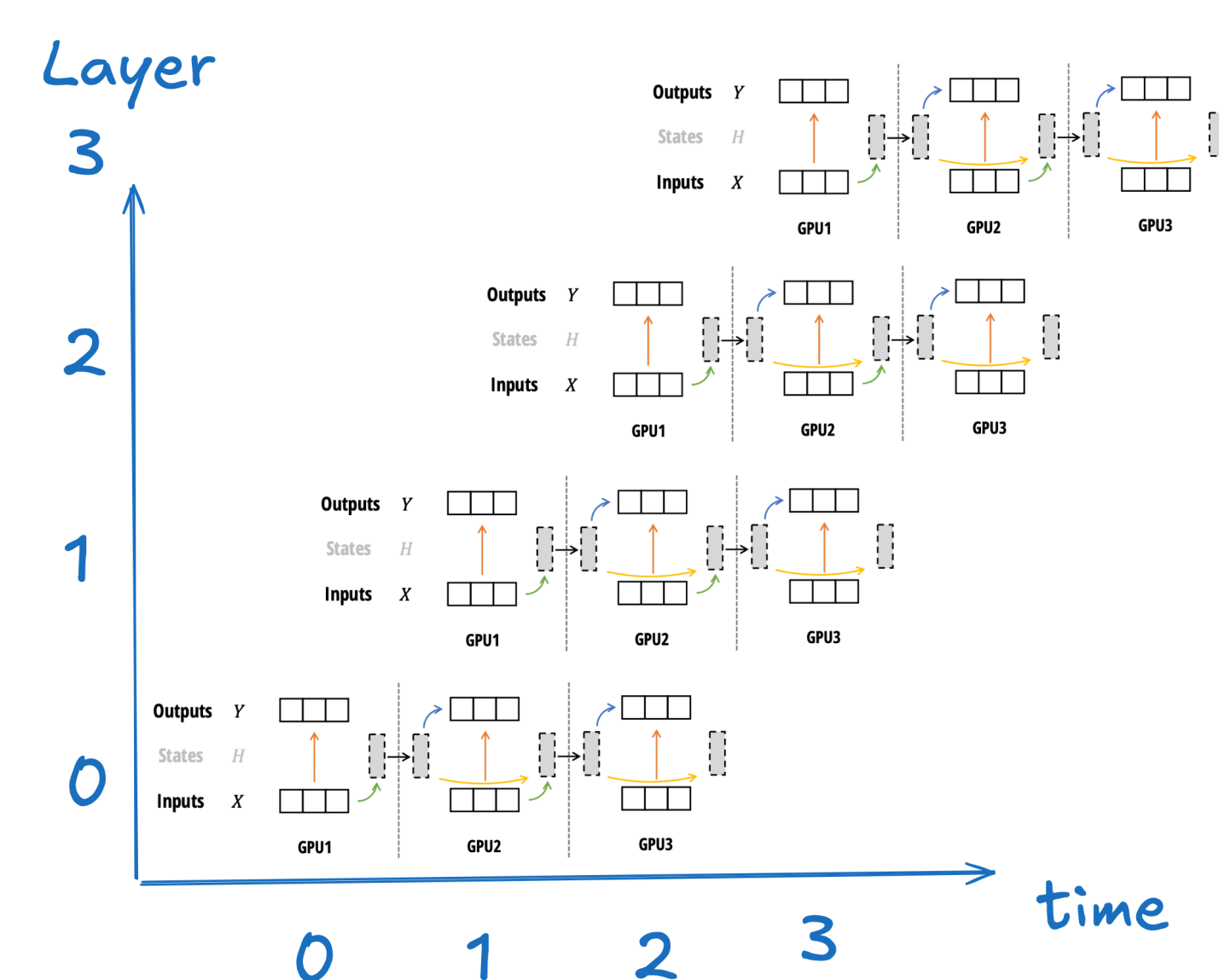
1. CP rank r asynchronously passes its final $K - 1$ tokens to rank $r+1$
2. Concurrently, ranks convolve their local tokens: $z = \text{causal_conv1d}(x)$
3. The leading outputs are corrected via the received tokens:
 $z[:K-1] = \text{causal_conv1d}(\text{cat}(x_{\text{recv}}, x[:K-1]))$

CP Recursion: State Passing

A straightforward CP implementation of Mamba2's recursion step comes from passing state across CP boundaries. This proceeds as follows:

1. Rank r solves the recursion locally and passes $z[-1]$ to rank $r + 1$.
2. Rank $r + 1$ waits on the passed state, then solves recursion locally.

The causal Mamba2 dependencies result in $\mathcal{O}(\text{seq_len})$ exposed communication. However, pipelining can be use to amortize this cost.



CP Recursion: Compute-Then-Correct

An alternative implementation which can have better strong-scaling properties is as follows:

Alternatively, a strategy similar to the CP convolution algorithm is also possible, in which we compute incorrect outputs with locally available tensors, and then correct the results via communication. GPUs never idle with this strategy.

In order to describe this strategy, we trade the global sequence index s for the pair of indices r, c with $r \in \{0, \dots, \text{cp_degree} - 1\}$ indexing the CP rank and $c \in \{0, \dots, \text{seq_len} / \text{cp_degree} - 1\}$ indexing the chunked sequence position. A valid CP implementation is as follows:

1. Every rank computes $\Sigma_r = \sum_c A_{rc}$: the sum of local gate values
2. Rank r asynchronously sends Σ_r to CP ranks $r' > r$
3. Every rank solves the recursion relation with its local inputs x_{rcd} and trivial initial state, producing (incorrect) final states y_{rd}^{final}
4. Rank r sends y_{rd}^{final} to CP ranks $r' > r$
5. Rank r computes its corrected initial state via $x_{rd}^{\text{initial}} = \sum_{r' < r} \exp(\Sigma_{r-1} + \dots + \Sigma_{r'+1}) y_{r'd}^{\text{final}}$
6. Every rank re-solves the recursion relation with its now-corrected initial states, yielding the correct outputs z_{rcd}

This strategy requires $\mathcal{O}(\text{seq_len} / \text{cp_degree})$ additional compute and has $\mathcal{O}(\text{cp_degree})$ exposed communication.

References

- [1] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality." [Online]. Available: <https://arxiv.org/abs/2405.21060>
- [2] "🔥 Flash Linear Attention." [Online]. Available: <https://github.com/flan-ai/flash-linear-attention>
- [3] P. Glorioso *et al.*, "Zamba: A Compact 7B SSM Hybrid Model." [Online]. Available: <https://arxiv.org/abs/2405.16712>
- [4] "Bamba: Inference-Efficient Hybrid Mamba2 Model 🐼." [Online]. Available: <https://huggingface.co/blog/bamba>
- [5] NVIDIA *et al.*, "Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models." [Online]. Available: <https://arxiv.org/abs/2504.03624>
- [6] "IBM Granite 4.0: hyper-efficient, high performance hybrid models for enterprise." [Online]. Available: <https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models>
- [7] "causal-conv1d." [Online]. Available: <https://github.com/Dao-AILab/causal-conv1d>